Simulation of Man-in-the-Middle Attack and Its Prevention

Asma Ashfaq | Information Security Project |

Roll No : BITF24M011

Made with GAMMA

Introduction

Every message sent over a network travels through routers, switches, and servers — and at any point, an attacker may be listening. One of the most deceptive threats is the Man-in-the-Middle (MITM) attack, where a malicious actor secretly positions themselves between two communicating parties.

The Problem

Network communication is not inherently secure — data can be exposed in transit.

The Threat

Attackers can intercept, read, and even modify messages without either party knowing.

This Project

We simulate a MITM attack in C++ to demonstrate how it works — and how to stop it.

What Is a Man-in-the-Middle Attack?

In a MITM attack, the attacker secretly intercepts the communication channel between a client and a server. Neither party realizes a third party is involved. The attacker can read every message exchanged and even modify the content before forwarding it — making the attack both powerful and invisible.

⚠ **Both the client and server believe they are communicating directly with each other — they never detect the attacker.**

Made with GAMMA

Problem Statement

Insecure communication channels create serious vulnerabilities. Because most users have no way to verify the integrity of a message in transit, attackers can silently exploit the connection.

Undetectable Interception

Victims have no indication that their messages are being read by a third party during transmission.

Data Theft

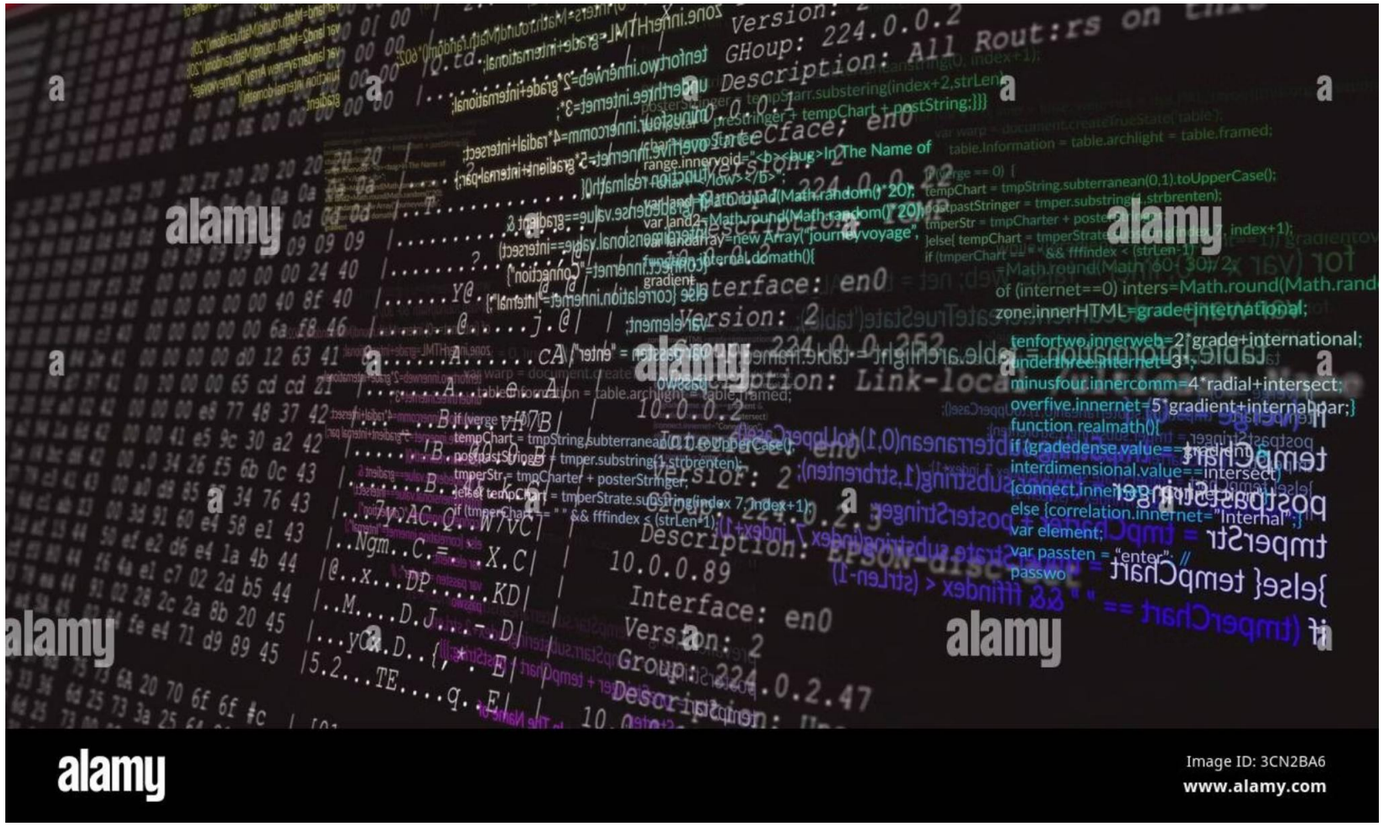
Sensitive information such as passwords, credentials, and personal data can be silently captured.

Message Tampering

Attackers can alter message content before it reaches the recipient, corrupting data integrity.

Made with GAMMA

Project Objectives



This project is designed to bring MITM concepts out of theory and into a working simulation. By building a real network scenario in C++, we can observe the attack — and its countermeasures — in action.

1 **Simulate the Attack**

Build a functional MITM simulation using C++ and TCP socket programming.

2 **Demonstrate Interception & Modification**

Show how an attacker captures and alters messages in real time.

3 **Apply Prevention via Encryption**

Demonstrate how encrypting messages renders interception useless.

Made with GAMMA

Tools & Technologies

C++

Core programming language used to build all three components — client, attacker, and server.

TCP Socket Programming

Enables real network communication between processes, replicating actual client-server interactions.

Winsock (Windows API)

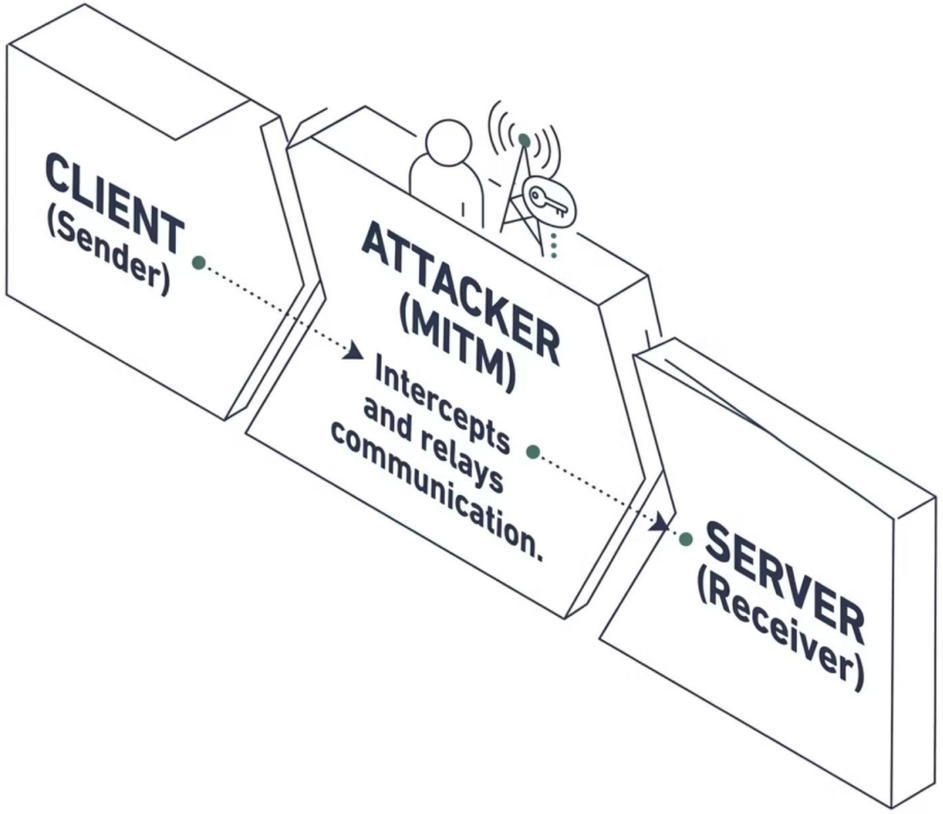
Windows-native socket library used to manage low-level network connections in the simulation.

VS Code / Visual Studio

Development environment for writing, compiling, and running the simulation programs.

Made with GAMMA

System Design



How It All Connects

The simulation runs three separate programs on the same machine, each communicating over TCP sockets:

Client — initiates connection and sends messages, unaware of the attacker
Attacker — listens on a port the client connects to, then relays traffic to the server
Server — receives and displays messages, believing they came directly from the client

Made with GAMMA

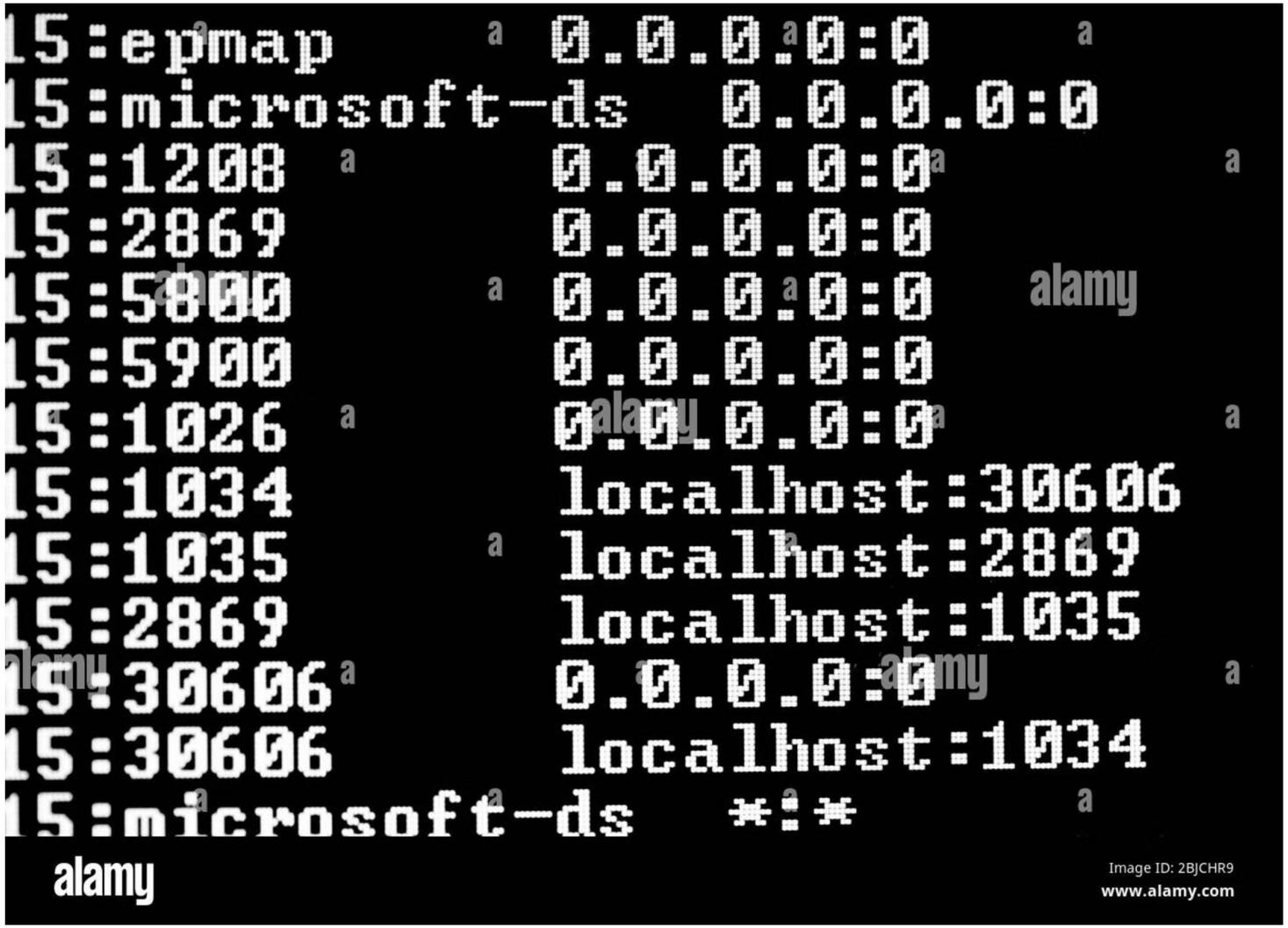
How the Attack Works (Without Encryption)

Without any encryption, every message travels as plain text. The attacker intercepts the message, appends a tamper marker, and forwards the modified version — all transparently. The server has no mechanism to detect the change.

⊗ **This illustrates why plaintext communication over untrusted networks is fundamentally insecure.**

Made with GAMMA

Demo Output



Observed Behavior in the Simulation

Actor	Message Seen
Client (Sent)	Hello
Attacker (Intercepted)	Hello
Server (Received)	Hello [HACKED]

The client sent the original message. The server received a modified version. Neither detected any anomaly — confirming the attack was successful.

Made with GAMMA

Prevention: Encryption

The most effective countermeasure against MITM attacks is encrypting messages before transmission. Even if the attacker intercepts the data, they cannot read or meaningfully alter it without the decryption key.

Without Encryption

Client sends: HelloAttacker sees: HelloServer receives: Hello [HACKED]

With Encryption

Client sends: Xy@#!! (encrypted)
Attacker sees: Xy@#!! (unreadable)
Server decrypts: Hello (safe)

Encryption ensures that even a successful interception yields no useful data to the attacker — protecting both confidentiality and integrity.

Made with GAMMA

Working / Demo

(Without Encryption):

```
PS C:\Users\User\Desktop\information security project code> .\client.exe
Enter message: hello
```

```
PS C:\Users\User\Desktop\information security project code> .\attacker.exe
Attacker waiting...
Intercepted: hello
```

```
PS C:\Users\User\Desktop\information security project code> .\server.exe
Server waiting...
Received: hello [HACKED]
Reply:
```

Made with GAMMA

With Encryption

```
PS C:\Users\User\Desktop\information security project code> .\client.exe
Enter message: hello
Reply: ok
Enter message:
```

```
PS C:\Users\User\Desktop\information security project code> .\attacker.exe
Attacker waiting...
Intercepted: #.''$
```

```
PS C:\Users\User\Desktop\information security project code> .\server.exe
Server waiting...
Received: hellok

Reply: ok
```

Made with GAMMA